

LECTURE 4

LLMOps: Serving, Deployment & Scaling Laws

vLLM · Docker · Autoscaling · Prometheus · Chinchilla · Cost Optimisation · Continuous Learning

AI-OP-IN-TH-000004-000009 · AI-OP-IN-CD-000002 · AI-PT-AD-TH-000001-000003 · AI-PT-AD-CD-000001

Learning Objectives

- 1 Compare FastAPI and vLLM serving frameworks — explain PagedAttention and continuous batching and when to use each
- 2 Design an LLM-specific Docker deployment: multi-stage build, CUDA base image, health check, volume-mounted weights
- 3 Describe horizontal scaling, load balancing (nginx/Traefik), and autoscaling triggers for LLM services
- 4 Build the observability stack: Prometheus metrics → Grafana dashboards → ClearML experiment tracking
- 5 Apply the Chinchilla result: $N \approx D$, explain how it overturned GPT-3 era training, and work a compute budget example
- 6 Calculate A100-hours and \$ cost for a 7B model training run using the FLOPs formula
- 7 Apply the cost optimisation decision tree: model selection, quantisation, semantic caching, tiered routing
- 8 Design a continuous learning system: drift detection → data collection → incremental LoRA → blue-green deployment

Where We Are — The Module 9 LLMOps Journey

Lecture 4 is the production operations capstone — it depends on everything built in previous Lectures

L2

MCP Deep Dive

Sub-Module 9.5

FastMCP · Tool registry · Security ·
Observability
You can build and operate MCP tool
servers in production

L3

A2A & Multi-Agent

Cross-module

A2A protocol · LangGraph ·
Coordination patterns · Alignment
You can design and implement multi-
agent workflows

L4

Serving, Deployment & Scaling Laws

Sub-Modules 9.3 + 9.4

vLLM · Docker · Autoscaling ·
Prometheus · Chinchilla · Cost
optimisation
You will know how to SERVE and
MONITOR models and optimise costs

After this Lecture you have the complete Module 9 picture: **build it → serve it (L4) → connect it to agents via MCP (L2) → orchestrate agents via A2A (L3)**

What Is LLM Serving — and Why Is It Hard?

Training a model is hard. Serving it to thousands of concurrent users reliably, cheaply, and quickly is harder.

LLM Serving: Running a trained LLM as a network service that accepts user requests, generates tokens, and returns results — **reliably, at scale, within latency SLAs, at predictable cost.**

Why 'just run model.generate()' breaks in production

1

One user at a time

naive model.generate() processes one request before starting the next. 100 concurrent users → 99 users are waiting in a queue. GPU sits idle most of the time.

2

Memory fragmentation

KV cache is allocated per request at full size even if the response is short. 70% of VRAM is wasted on padding. You serve 3 users where hardware supports 12.

3

No latency control

Long requests block short ones. A user asking 'What is 2+2?' waits behind a user generating a 10,000-token document. p99 latency is unpredictable.

4

No reliability

Process crashes silently. No health checks. No automatic restart. No circuit breakers. One GPU OOM event → entire service down with no alerting.

5

No cost visibility

You have no idea which prompts cost 1 cent and which cost \$1. No token counts. No per-user cost attribution. Budget overruns are invisible until the AWS bill arrives.

This lecture is about solving all 5 of these problems — vLLM, Docker, autoscaling, and Prometheus are the tools; understanding the problems is why they exist.

The LLM Serving Tooling Landscape — What Is What?

Beginners encounter many names — vLLM, FastAPI, HuggingFace, TGI, Docker, Prometheus — here is how they fit together

 vLLM Inference Engine Open-source LLM inference and serving library. NOT a company — a UC Berkeley research project, now widely adopted. Does: Implements PagedAttention + continuous batching. Exposes an OpenAI-compatible REST API. The production standard for serving open-weight models. <i>Not for: Building custom business logic — that is FastAPI's job. vLLM handles only the model inference layer.</i>	 FastAPI Web Framework Python web framework for building REST APIs. The same tool used for any Python web service — not LLM-specific. Does: Wraps HuggingFace model.generate() in an HTTP endpoint. Gives you full control over request handling, auth, rate limiting. <i>Not for: High-traffic production serving — no PagedAttention, no continuous batching. Use for prototyping or low-traffic custom endpoints.</i>	 HuggingFace Transformers Model Library Python library for loading and running pre-trained models. The standard for accessing open-weight models (Llama, Mistral, Falcon). Does: Loads model weights, provides tokenisers, exposes model.generate(). Used by both FastAPI (directly) and vLLM (under the hood). <i>Not for: Production serving at scale — model.generate() is single-threaded and ignores batching opportunities.</i>
 Docker Container Runtime Container technology that packages an application + all its dependencies into a portable image. Not LLM-specific at all. Does: Ensures your vLLM server runs identically on your laptop, staging VM, and production A100 cluster. Manages CUDA versions. <i>Not for: Actually running the model — that is vLLM's job. Docker is the deployment shell, not the inference engine.</i>	 Prometheus + Grafana Observability Stack Prometheus: metrics collector. Grafana: dashboard visualiser. Industry-standard monitoring stack used across all software. Does: Scrapes /metrics from your vLLM/FastAPI server. Tracks token throughput, latency, error rate, GPU utilisation. Triggers alerts. <i>Not for: Improving inference speed — monitoring tells you what is slow, but vLLM optimisations fix it.</i>	 ClearML ML Experiment Tracker ML experiment and model lifecycle management platform. Tracks training runs, model versions, and deployment metadata. Does: Logs training curves, eval benchmarks, model artifacts. Provides model registry. Connects to Prometheus for deployment monitoring. <i>Not for: Real-time serving metrics — that is Prometheus. ClearML manages the ML development lifecycle.</i>

In production: vLLM handles the inference · Docker packages it · FastAPI adds business logic · Prometheus observes it · ClearML tracks the ML lifecycle

What Are Scaling Laws — and Why Should a Practitioner Care?

Scaling laws are empirical rules that predict model quality from compute budget, model size, and data size

Scaling law: A mathematical relationship between model size (N parameters), training data (D tokens), compute budget (C FLOPs), and model quality (loss). Discovered empirically — not derived from theory.

Three things scaling laws tell us (and why each matters to you)

Bigger models = better quality

(Kaplan et al., 2020 — GPT-3 era)

What it says: You can predict how much better a model gets when you 10× its parameters. Log-linear relationship: quality improves predictably with scale.

For practitioners: Helps you choose model size for your budget. A 7B model is not 7× better than a 1B model — the relationship is logarithmic.

Limitation: Kaplan's rule was wrong about one thing: it ignored the training data side. GPT-3 was severely undertrained.

Data matters as much as model size

(Chinchilla — Hoffmann et al., 2022)

What it says: For a given compute budget, the optimal strategy is N parameters ≈ D training tokens. GPT-3 (175B, 300B tokens) was training-data-starved.

For practitioners: If you are choosing between 70B trained on 1T tokens vs. 7B trained on 15T tokens — the 7B wins for your inference budget.

Limitation: Compute-optimal training is optimal for training cost, not for serving cost. Inference-optimal is a separate calculation.

You can predict exact training cost

(Chinchilla + FLOPs formula)

What it says: $C \approx 6 \times N \times D$ FLOPs. Given an A100 at 312 TFLOPs, you can calculate exact GPU-hours and \$ to train any model to any quality target.

For practitioners: Before you run a single training step, you can budget the full cost. No more 'we ran out of compute halfway through' surprises.

Limitation: The 6× coefficient is approximate and varies by architecture. Use as a planning estimate, not an invoice.

You will not pre-train a frontier model. But you WILL choose between model sizes, negotiate training budgets, and decide when self-hosting beats API pricing — scaling laws drive all three.

What You Will Learn and Why It Matters

Two halves — each a complete discipline in its own right — both essential for production LLM systems

HALF 1 — Serving & Deployment

S1

FastAPI vs vLLM

Choose the right serving stack. Know what PagedAttention and continuous batching actually are — not just buzzwords.

S2

PagedAttention — How It Works

The innovation that makes vLLM 10–24× faster. Understand it mechanically, not just the claim.

S3

Docker for LLMs

Package any LLM service for reliable deployment anywhere. CUDA base images, volume-mounted weights, health checks.

S4

Scaling, Load Balancing & Autoscaling

Handle traffic spikes without paying for idle capacity. Know which autoscaling trigger to use and why.

S5

Monitoring Stack + Production Pipeline

6 LLM-specific metrics. Hallucination drift detection. 6-step blue-green deployment pipeline.

HALF 2 — Scaling Laws & Cost

S6

Chinchilla Scaling Laws

The result that overturned how the entire industry trains models. $N \approx D$ and what it means for your model choices.

S7

Compute-Optimal vs Inference-Optimal

Why Llama 3.1 8B was deliberately overtrained — and why that is the right decision for production APIs.

S8

Learning Rate Schedules

The first-class hyperparameter. Bad LR = training failure. Cosine decay, WSD, and the LR range test.

S9

Cost Optimisation Decision Tree

6 levers that compound. Self-hosting break-even. API vs self-host cost table across 3 traffic tiers.

S10

Continuous Learning & Model Registry

Production models degrade. Drift detection → incremental LoRA → blue-green → rollback. Design it from day one.

After this lecture you can deploy an LLM service, keep it running reliably, and decide whether building or buying makes financial sense.

SECTION 1

Serving & Deployment

FastAPI · vLLM · Docker · Scalability · Monitoring stack

Serving Frameworks: FastAPI vs vLLM

Two dominant approaches — different architectures, very different throughput ceilings

FastAPI + HuggingFace

- Flexible Python web server wrapping HuggingFace model.generate()
- Full control over pre/post-processing, authentication, business logic
- Static batching: waits for batch to fill before processing — GPU idles
- No built-in KV cache management — naive memory allocation
- Best for: low-traffic (<100 req/min), research, custom logic
- Throughput: 5–20 req/min for 7B model on A100

vLLM (PagedAttention + Continuous Batching)

- Purpose-built LLM serving engine — production standard in 2024–2026
- PagedAttention: manages KV cache in non-contiguous memory pages
- Continuous batching: insert new requests as slots free up — GPU always busy
- OpenAI-compatible REST API — drop-in replacement for OpenAI API clients
- Best for: production traffic (100–10,000+ req/min), cost-sensitive deployments
- Throughput: 10–24× vs. naive FastAPI serving for same hardware

```
# FastAPI pseudo-pattern
POST /generate
input = request.prompt
tokens = tokenise(input)
output = model.generate(tokens,
    max_new_tokens=512)
RETURN detokenise(output)
```

```
# vLLM pseudo-pattern
# Same API — engine handles batching
POST /v1/completions
{ model, prompt, max_tokens }
# vLLM queues internally
# PagedAttention manages KV cache
STREAM tokens as generated
```

Decision rule: prototype/custom logic → FastAPI · production serving with concurrent users → always vLLM

PagedAttention — How vLLM Manages KV Cache Memory

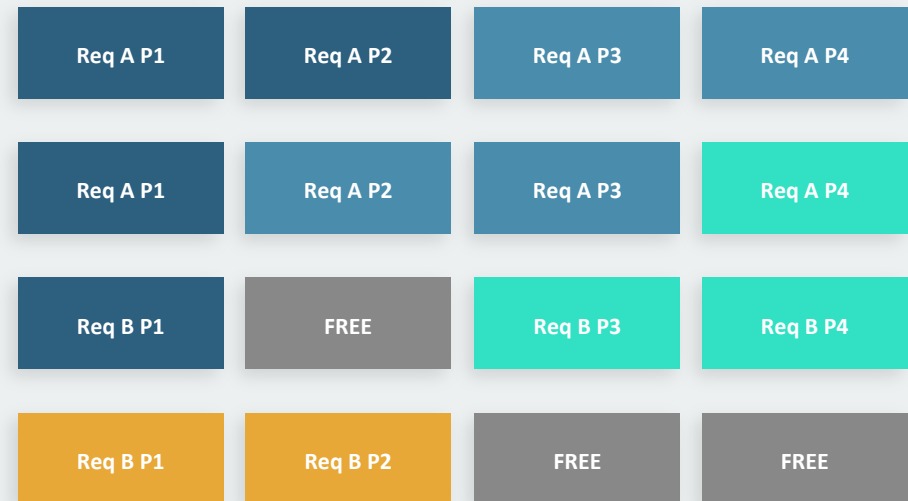
The key innovation behind vLLM's 10–24× throughput gain over naive serving

Problem: Naive KV cache allocation



VRAM fragmentation — reserved but unused space = wasted capacity

Solution: PagedAttention — virtual memory pages



Pages for Req A and B are interleaved in any free slot — zero wasted VRAM

✓ 3–4× more concurrent requests per GPU

✓ Enables 128K+ token context windows

✓ Zero fragmentation overhead

✓ Used by: vLLM, TGI, TensorRT-LLM

PagedAttention = OS virtual memory paging applied to KV cache — eliminated the #1 memory waste in LLM serving

Docker for LLM Services — AI-Specific Patterns

Why containerise · LLM-specific Dockerfile · Docker Compose stack

Why Docker is non-negotiable for LLM services

Dependency isolation

CUDA version, Python, transformers, torch must all match exactly. Docker freezes the entire stack.

Reproducibility

Same container runs on dev laptop, staging, and production cloud. Eliminates 'works on my machine'.

Independent scaling

Scale the vLLM container without touching the API gateway or MCP sidecar. Each service scales separately.

CI/CD ready

Git push → Docker build → security scan → push registry → deploy. Same pipeline as model serving layer.

LLM-specific Dockerfile — pseudo-steps

```
● ● ●

# Stage 1: Build
BASE nvidia/cuda:12.1-devel-ubuntu22.04
INSTALL python3, pip, build tools
COPY requirements.txt
RUN pip install vllm torch transformers

# Stage 2: Runtime (smaller image)
BASE nvidia/cuda:12.1-runtime-ubuntu22.04
COPY --from=build /usr/local/lib .
COPY server.py /app/
VOLUME /models # weights mounted - not baked in
EXPOSE 8000 # vLLM API
HEALTHCHECK CMD curl /health || exit 1
CMD vllm serve /models/llama-3-8b
```

Docker Compose stack topology

```
● ● ●

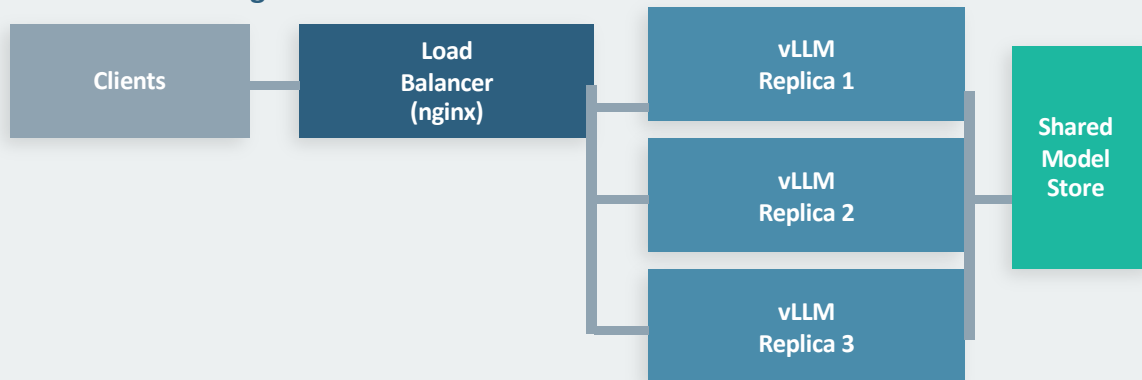
services:
  vllm:
    image: vllm-server:latest
    volumes: [./models:/models]
    ports: [8000:8000]
    deploy: { resources: { gpus: all } }
  nginx-lb:
    depends_on: [vllm]
  redis:
    # semantic cache store
  prometheus:
    # scrape /metrics on all services
  grafana:
    # dashboards from prometheus
```

Volume-mount model weights — never bake them into the image. Images stay small (< 5 GB). Weights are mounted at runtime from shared NFS/S3

Scalability & Load Balancing for LLM Services

Horizontal scaling · nginx/Traefik routing · Autoscaling triggers · Tensor vs Pipeline parallelism

Horizontal scaling architecture



Autoscaling triggers — when to scale out vs. in

Trigger metric	Scale OUT threshold	Scale IN threshold
GPU utilisation (%)	GPU util > 75% for 3 min	GPU util < 30% for 10 min
Request queue depth	Queue > 50 pending reqs	Queue = 0 for 5 min
p99 latency (ms)	p99 > 800ms	p99 < 200ms for 10 min
Cost per request (\$)	— (informational)	Cost above budget threshold

Multi-GPU parallelism strategies

Tensor Parallelism

Split weight matrices across GPUs (attention heads). All GPUs process every token. Requires NVLink (fast interconnect). Best for single-request latency.

Pipeline Parallelism

Assign consecutive layers to different GPUs. GPU 1 processes layers 1–16; GPU 2 processes layers 17–32. Lower interconnect bandwidth needed. Best for throughput.

Data Parallelism

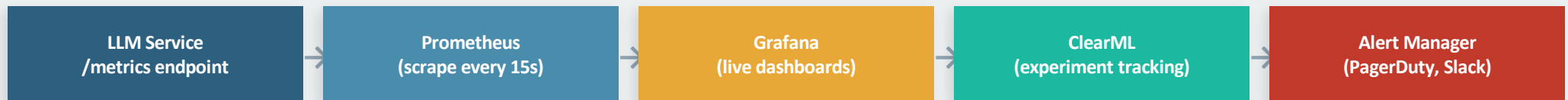
Replicate full model on N GPUs; each handles a different batch. Independent replicas — no inter-GPU communication. Scales throughput linearly.

Stateless serving: LLM replicas hold no user state — load balancer routes round-robin. Session affinity needed only for very long multi-turn conversations

LLM Monitoring & Observability Stack

Prometheus → Grafana → ClearML · LLM-specific metrics · Alerting rules

Observability data flow



LLM-specific metrics — beyond standard web service metrics

Metric	What it measures	Alert threshold	Why it matters
Hallucination rate	% of responses containing factual errors detected by evaluator	Rise > 2% week-over-week	Signals model quality degradation / prompt drift
Token throughput (tok/s)	Tokens generated per second across all requests	Drop > 20% vs. baseline	Detects performance regression from config change
p99 generation latency	99th-percentile time to complete a response	p99 > 3s	Impacts user experience; may trigger autoscale
Cost per 1K tokens (\$)	Actual inference cost including GPU amortisation	Exceeds budget × 1.2	Cost overrun early warning; triggers tiered routing
Cache hit ratio	Fraction of requests served from semantic cache	Drops < 0.3	Cache misconfiguration or prompt distribution shift
Context window utilisation	Average tokens per request / max context	> 90% sustained	Approaching limit; may need chunking or summarisation

Standard web metrics (CPU, memory, error rate) are necessary but not sufficient for LLM services — you need output-quality metrics too

Production Deployment Pipeline — End-to-End

From model checkpoint to monitored production service in 7 steps

1 CHECKPOINT VALIDATION

Run eval benchmarks (MMLU, HumanEval, HellaSwag) on checkpoint. Compare perplexity vs. base model. Must meet quality gates before proceeding.

2 QUANTISE & OPTIMISE

Apply AWQ INT4 quantisation. Run vLLM with FlashAttention-2. Benchmark throughput (tok/s) and p99 latency on representative prompt set.

3 DOCKER BUILD & SCAN

Multi-stage build. Tag with git SHA and model version. Run Trivy/Snyk security scan — fail pipeline on high-severity CVEs. Push to registry.

4 STAGING DEPLOY

Deploy to staging stack (1 GPU replica). Run smoke tests: health check, tools/list (if MCP), 100-prompt quality sample. Check all 6 metrics emitting.

5 LOAD TEST

Simulate 10× expected peak traffic. Verify p99 latency SLA at load. Check autoscaler fires correctly. Confirm GPU utilisation 70–80% target.

6 BLUE-GREEN SWAP

Route 10% traffic to new (green) deployment. Compare hallucination rate, cost/query, p99 vs. current (blue). Ramp to 50% → 100% if metrics hold.

Blue-green deployment is the safety net — the old version keeps running until the new version proves itself at partial traffic

SECTION 2

Scaling Laws & Cost Optimisation

Chinchilla · Compute budgets · Cost decision tree · Continuous learning

Chinchilla Scaling Laws — The Compute-Optimal Result

Hoffmann et al. 2022 — the paper that reshaped how every major lab trains models

Chinchilla result: For a given compute budget C (FLOPs), the optimal model has N parameters and D tokens where $N \approx D$ (parameters $\approx$ training tokens) · **Optimal: $D = 20 \times N$**

How Chinchilla overturned the GPT-3 / Kaplan era

Model	Parameters (N)	Training tokens (D)	D / N ratio	Verdict
GPT-3 (Kaplan era)	175B	300B	1.7×	SEVERELY UNDERTRAINED
Gopher	280B	300B	1.1×	UNDERTRAINED
Chinchilla	70B	1.4T	20×	COMPUTE-OPTIMAL ✓
Llama 3.1 8B	8B	15T	1,875×	INFERENCE-OPTIMAL (deliberate overtrain)

Practical formulae — worked example for a 7B model



Optimal tokens for 7B model: $D = 20 \times N = 20 \times 7B = 140B$ tokens
FLOPs required: $C \approx 6 \times N \times D = 6 \times 7 \times 10^9 \times 140 \times 10^9 \approx 5.9 \times 10^{21}$ FLOPs
A100 delivers: $312 \text{ TFLOPs BF16} \rightarrow 312 \times 10^{12} \text{ FLOPs/s}$
A100-hours needed: $C / (312 \times 10^{12} \times 3600) \approx 5,271 \text{ A100-hours}$
Cost at \$2/hr spot: $\approx \$10,500$ to train a compute-optimal 7B model

Key insight: Chinchilla (70B, 20× tokens) outperformed Gopher (280B, 1× tokens) on nearly every benchmark — bigger is not always better if undertrained

Compute-Optimal vs Inference-Optimal Training

The deliberate overtrain strategy — why Llama 3.1 8B used 1,875× more tokens than Chinchilla says

Compute-Optimal (Chinchilla)

- Minimise training FLOPs for a given performance target
- Use the largest model that fits the compute budget
- Train for exactly the Chinchilla-optimal token count
- Result: best model quality per training dollar
- Problem: the model may be expensive to serve (many parameters)
- Example: 70B model trained on 1.4T tokens

Inference-Optimal (Deliberate Overtrain)

- Minimise per-query inference cost over the model's lifetime
- Use a smaller model and train it on far more tokens
- Train well beyond Chinchilla-optimal — spend extra training FLOPs
- Result: cheaper to serve, comparable quality
- Only makes sense when lifetime query volume is very large
- Example: Llama 3.1 8B on 15T tokens — 1,875× Chinchilla optimal

When to choose which strategy

Situation	Choose	Rationale
Research model, few queries	Compute-optimal	Maximise quality; serve cost doesn't matter much
Production API, millions of queries/day	Inference-optimal	Serving cost dominates; small model pays off at scale
Edge deployment (phone/laptop)	Inference-optimal + extreme overtrain	Must fit in 4 GB VRAM; quality is secondary to size
Fine-tuning a base for domain tasks	Inference-optimal base	Start from overfit small model; fine-tune cheaply

Llama 3.1 8B on 15T tokens = Meta spent extra training compute so that you spend less serving compute — the cost is borne once, the saving is reaped trillions of queries

Learning Rate Schedules — The First-Class Hyperparameter

Warmup · Cosine decay · WSD schedule · Linear scaling rule · LR range test

Linear Warmup → Cosine Decay

Start at near-zero LR → linearly increase to peak over first 1–2% of steps (warmup) → cosine curve to ~10% of peak (decay). Standard for most LLM training runs.

When: Use when: token budget is known in advance. Cosine decay must end at the planned token count.

Cosine with Restarts (SGDR)

Periodic resets of LR back to peak. Each restart allows the optimiser to escape local minima. Produces multiple checkpoints — good for model ensembling.

When: Use when: you want to explore the loss landscape or produce ensemble checkpoints.

WSD (Warmup-Stable-Decay)

Long stable phase at peak LR after warmup, then a rapid linear decay at the very end. Used by MiniCPM and several 2024 models for very large corpora.

When: Use when: training on very large datasets (>1T tokens) where long stable phase is important.

Key rules for LR tuning

Linear scaling rule

When you double the batch size, double the peak LR. Keeps gradient variance constant.

LR range test (Smith 2017)

Sweep LR from $1e-7$ to 1 over ~100 steps. Plot loss vs. LR. Pick LR just before loss diverges.

Warmup length

1–2% of total training steps. Too short → early instability. Too long → wasted compute at low LR.

Min LR at decay end

Typically 10% of peak. Don't decay to zero — tiny LR still allows late-stage refinement.

Bad LR choice is the single most common cause of training failure — too high → divergence; too low → slow convergence; wrong schedule → premature plateau

Cost Optimisation Decision Tree

Six levers for reducing LLM inference cost — apply in this order

1 Model Selection

Use the smallest model that meets quality requirements. A well-prompted 8B model often matches 70B for narrow tasks at 1/10th the cost.

```
USE small_model # 10x cheaper
ELSE: evaluate medium_model
```

2 Quantisation

INT4/INT8 reduces VRAM 4x → fewer GPUs needed → lower serving cost. Less than 1% quality degradation on most tasks.

```
# 4.5 GB vs 14 GB — 3x fewer GPUs needed
```

3 Semantic Caching

Cache LLM responses for semantically similar queries using embedding similarity. Can reduce API calls 30–70% for FAQ-style applications.

```
cached = cache.get(embed, threshold=0.95)
IF cached: RETURN cached # zero LLM cost
```

4 Continuous Batching

Group requests to increase GPU utilisation. vLLM continuous batching automates this — 10–24x throughput from same hardware.

```
# result: GPU utilisation 20% → 85%
```

5 Prompt Compression

LLMLingua compresses long prompts to 20–30% of original size while preserving meaning. Fewer tokens → lower cost proportionally.

```
ratio=0.3) # 70% token reduction
```

6 Tiered Routing

Binary classifier routes simple queries to small/cheap models and complex queries to large/expensive models automatically.

```
IF complexity == 'simple': USE 8B # cheap
ELSE: USE 70B # quality
```

Apply in order: selection → quant → cache → batching → compression → routing. Each lever compounds — combined savings can exceed 90% cost reduction

Cost Comparison: API vs Self-Hosted at Different Traffic Levels

GPT-4o · GPT-4o-mini · Self-hosted Llama 3 8B — monthly cost at 3 traffic tiers

Assumptions: average 1,000 tokens input + 500 output per request · A100 spot \$2/hr · 14 GB VRAM for Llama 3 8B FP16

Model	Cost per 1M tokens	10K req/day (~150M tok/mo)	100K req/day (~1.5B tok/mo)	1M req/day (~15B tok/mo)	When to use
GPT-4o	\$15 (in) + \$60 (out)	~\$13,500/mo	~\$135,000/mo	~\$1.35M/mo	Complex reasoning, code, analysis; low volume
GPT-4o-mini	\$0.15 (in) + \$0.60 (out)	~\$135/mo	~\$1,350/mo	~\$13,500/mo	Medium tasks; startup budgets; test quickly
Llama 3 8B (self-host, FP16)	~\$0.05 (infra cost)	~\$45/mo	~\$450/mo	~\$4,500/mo	High-volume; narrow domain tasks; privacy
Llama 3 8B (INT4, 2x throughput)	~\$0.025	~\$22/mo	~\$225/mo	~\$2,250/mo	Best cost at scale — quantise and serve

Break-even: when does self-hosting beat GPT-4o-mini?

● ● ● Self-host monthly infra cost: \$450 (A100 spot × 24h × 30d × utilisation)
GPT-4o-mini at 100K req/day: \$1,350/month
Break-even: ~33K requests/day → self-host wins above this volume
Rule: < 10K req/day → API · > 50K req/day → self-host · 10-50K req/day → benchmark both

These costs change rapidly — re-run this analysis quarterly. GPU spot prices and API pricing both shift significantly year over year

Continuous Learning Systems

Production models degrade over time — drift detection → retrain → deploy safely

The continuous learning lifecycle

1

DRIFT DETECTION

Monitor hallucination rate, user feedback thumbs-down rate, and output perplexity. Detect statistically significant degradation vs. baseline using Mann-Whitney U test or CUSUM.

2

DATA COLLECTION

Log production inputs and outputs. Human review queue for flagged outputs. Filter for high-quality examples (correct answers confirmed). Dedup against training set.

3

INCREMENTAL FINE-TUNE

QLoRA on new data with data mixing (80% new, 20% original training data). Mixing prevents catastrophic forgetting. Eval on held-out benchmark before promoting.

4

SHADOW EVALUATION

Run new model on last 1,000 production queries in shadow mode. Compare hallucination rate, cost, and latency vs. current model. Must beat current on all three metrics.

5

BLUE-GREEN DEPLOY

10% → 50% → 100% traffic. Monitor metrics at each stage for 24h before ramping. Rollback to previous version if any metric regresses > threshold.

Model registry (MLflow / ClearML): every checkpoint tracked with eval metrics, deployment metadata, data lineage — enables reproducible rollback to any version

Project — A Complete Deliverable Map

All 5 deliverables mapped to Lectures of the module

D1

Pre-Training Warm-up

- Train a small CLM (~120M params) on a domain corpus — pseudo-code walkthrough
- Save checkpoint; confirm loss convergence in ClearML
- Log training curves; verify CLM vs. MLM objective difference in loss

D2

Optimised Inference

- Load Llama 3 8B with AWQ INT4 quantisation
- Benchmark throughput: vLLM continuous batching vs. naive HuggingFace
- Produce throughput comparison table: tok/s, VRAM, p99 latency

D3

Production Deployment

- Containerise vLLM endpoint (Dockerfile + health check + Prometheus metrics)
- Launch Docker Compose: vLLM + Redis + Prometheus + Grafana
- Demonstrate all 6 LLM observability metrics on Grafana dashboard

D4

MCP Integration

- Deploy FastMCP server as Docker sidecar (3 tools)
- Agent: tools/list discovery → tools/call chain → JSON-RPC verified
- Test 2-tool chain query; verify trace in logs

D5

Cost Analysis

- Cost/1M tokens: GPT-4o vs. mini vs. self-hosted Llama 3 8B vs. INT4
- Apply cost governance: per-tool rate limiting + session budget
- 1-page production readiness report with justified architecture recommendation

Deliverable: GitHub repo with reproducible Docker Compose · training scripts · benchmark results · 1-page production readiness report

LLMOps: Serving, Deployment & Scaling Laws

- ① FastAPI = flexible, low-traffic, custom logic. vLLM = production, PagedAttention + continuous batching = 10–24× throughput.
- ② PagedAttention: manages KV cache in non-contiguous pages — eliminates VRAM fragmentation, enables 128K+ context, 3–4× more concurrent requests.
- ③ Docker: CUDA base image, volume-mounted weights (never baked in), health check, Prometheus /metrics. Multi-stage build = small runtime image.
- ④ Observability: 6 LLM-specific metrics (hallucination rate, tok/s, p99, cost/query, cache hit, context utilisation) on top of standard web metrics.
- ⑤ Chinchilla: $N \approx D$ (optimal $N \times 20$ tokens). GPT-3 era = severely undertrained. Llama 3.1 8B overtrained deliberately for cheap inference.
- ⑥ Cost optimisation order: model selection → quantisation → semantic caching → batching → prompt compression → tiered routing. Savings compound.
- ⑦ Break-even for self-hosting: > ~33K req/day vs. GPT-4o-mini. INT4 quantisation roughly doubles throughput, halves serving cost.
- ⑧ Continuous learning: drift detection → data collection → incremental LoRA + data mixing → shadow eval → blue-green deploy → model registry.

Module 9 complete: Pre-Training → Infrastructure → Serving → Scaling Laws → MCP in Production → Mini Project 9